Cuestionario 1er parcial de Programación IV

1. ¿Qué es un paradigma de programación? ¿Qué caracteriza al paradigma de Programación Orientada a Objetos (POO)?
2. ¿Qué es una clase y qué es un objeto? Explique la diferencia entre ambos mediante un ejemplo.
3. ¿Cuáles son los cuatro pilares fundamentales de la Programación Orientada a Objetos? Explique brevemente cada uno:

- Abstracción.
- Encapsulamiento.
- Herencia.
- Polimorfismo.

4. ¿Cuáles son los atributos y los métodos de una clase? ¿En qué se diferencian?

5. ¿Es lo mismo un programador que un desarrollador de software? Explique las diferencias entre ambos perfiles.

6. ¿Qué es un desarrollador Full Stack? ¿Qué conocimientos y responsabilidades implica este rol?

7. ¿Qué es el Frontend y qué es el Backend? ¿Cuál es la función de cada uno dentro de una aplicación web?

8. ¿Qué es un Diagrama de Flujo de Datos (DFD) y por qué lo utilizamos durante el análisis de un sistema? ¿Qué elementos principales componen un DFD? Explique la función de cada uno.

9. ¿Por qué es importante diagramar una base de datos antes de comenzar su implementación?

10. ¿Qué es un modelo entidad-relación (MER) y qué elementos lo componen? ¿y el modelo relacional?

11. ¿Qué es un mockup y para qué se utiliza en el desarrollo de software?

12. ¿Por qué es importante realizar el análisis y diseño de un sistema antes de comenzar a programar?

13. README.md: ¿Qué es un archivo README y cuál es su función dentro de un repositorio de GitHub? ¿Qué información debería incluir la documentación de un proyecto?

14. Estrategias SEO: ¿Qué es el SEO y cuál es su importancia en el desarrollo web? Mencione algunas estrategias que podemos implementar desde HTML para mejorar el posicionamiento de una página en los motores de búsqueda.

15. Git vs. GitHub: ¿Cuál es la diferencia entre Git y GitHub? ¿Por qué se complementan?

16. ¿Qué es un repositorio local y qué es un repositorio remoto? ¿Qué función cumple cada uno?

17. ¿Qué es un commit y por qué es importante realizar commits frecuentes con mensajes descriptivos?

18. ¿Qué es una rama en Git y qué ventajas ofrece trabajar con diferentes ramas dentro de un mismo proyecto?

19. ¿Qué es un merge y en qué situaciones necesitamos realizarlo?

20. ¿Qué es un Pull Request y cuál es su importancia en el trabajo colaborativo? ¿Qué se debe tener en cuenta antes de solicitar una revisión?

21. Explicar la función de los siguientes comandos:

- git clone
- git add
- git commit
- git push
- git pull
- git merge
- git status

22. ¿Por qué es importante mantener actualizado nuestro repositorio local antes de comenzar a trabajar en una tarea? ¿Qué problemas podemos evitar?

23. Buenas prácticas: ¿Qué estrategias aplicarías para organizar el trabajo de un equipo de desarrollo utilizando Git y GitHub? Menciona la importancia de las ramas, los commits descriptivos, los Pull Requests y la revisión de código.

24. Estructura HTML: ¿Cuál es la estructura básica de un documento HTML y qué función cumplen `html`, `head` y `body`?

25. Etiquetas semánticas: ¿Qué son las etiquetas semánticas de HTML? Menciona ejemplos y explica por qué es importante utilizarlas.

26. ¿Qué función cumplen elementos como `input`, `label`, `select`, `textarea` y `button`?

27. ¿Cómo se crea un enlace en HTML? ¿Qué diferencia existe entre un enlace interno y uno externo?

28. ¿Cómo se incorpora una imagen en HTML y qué importancia tiene el atributo `alt`?

29. ¿Cómo se genera una lista ordenada y una desordenada?

30. CSS: ¿Cuáles son las tres formas de incorporar CSS a un documento HTML? ¿Cuál recomendarías para un proyecto y por qué?

31. ¿Qué es y cómo se trabaja una variable en CSS?

32. ¿Qué es el Box Model de CSS? Explica qué representan `content`, `padding`, `border` y `margin`.

33. ¿Para qué sirve Flexbox y qué tipo de problemas de diseño permite resolver?

34. CSS Grid: ¿Qué es CSS Grid y en qué situaciones resulta conveniente utilizarlo?

35. Responsive Design: ¿Qué significa que una página sea responsive? ¿Por qué es importante que un sitio pueda adaptarse a diferentes tamaños de pantalla?

35. Media Queries: ¿Para qué sirven las Media Queries en CSS? Menciona un ejemplo de situación en la que serían necesarias.

36. Bootstrap: ¿Qué es Bootstrap y para qué podemos utilizarlo en un proyecto HTML/CSS?

37. Bootstrap vs CSS puro: Si una característica puede resolverse correctamente utilizando una clase de Bootstrap, ¿por qué podría ser innecesario escribir una gran cantidad de CSS personalizado para conseguir el mismo resultado?

38. ¿Qué función cumple **`!important`** en CSS y por qué su uso excesivo puede considerarse una mala práctica?

39. ¿Qué es JSON y para qué se utiliza en el desarrollo web?

40. ¿Qué es una arquitectura de diseño de software? Explique en qué consiste la arquitectura basada en componentes y la estructura modular.

41. ¿Qué son las clases container, row y col? Explique cómo se relacionan entre sí.

42. ¿Qué son los breakpoints en Bootstrap y para qué se utilizan?

43. ¿Cómo se puede incorporar Bootstrap a un proyecto web? Explique las diferencias entre utilizar un CDN e instalarlo mediante npm.

44. ¿Qué es JavaScript y para qué se utiliza en el desarrollo web?

45. ¿Cuál es la diferencia entre HTML, CSS y JavaScript?

46. ¿Qué es una variable en JavaScript? ¿Qué diferencias existen entre `let`, `const` y `var`?

47. ¿Qué tipos de datos básicos existen en JavaScript? Mencione algunos ejemplos.

48. ¿Qué son las funciones en JavaScript y para qué se utilizan?

49. ¿Qué son las estructuras condicionales en JavaScript? Explique el uso de if, else y else if.

50. ¿Qué son los bucles o estructuras repetitivas? Explique el funcionamiento de for y while.

51. ¿Qué es un array en JavaScript? ¿Para qué se utiliza?

52. ¿Qué es el DOM y cómo permite JavaScript interactuar con una página web?

53. ¿Qué es un evento en JavaScript? Mencione algunos ejemplos.

54. ¿Qué significa manipular el DOM? Mencione algunas operaciones que pueden realizarse mediante JavaScript.

55. ¿Qué es React y para qué se utiliza en el desarrollo web?

56. ¿Qué es un componente en React y qué función cumple?

57. ¿Qué diferencias existen entre un componente y una página en React?

58. ¿Qué es JSX y qué ventajas ofrece al trabajar con React?

59. ¿Qué son las props en React y para qué se utilizan?

60. ¿Qué es el hook `useState`? Explique cómo se utiliza.

61. ¿Qué es el hook `useEffect` y en qué situaciones puede utilizarse?
62. ¿Cómo se comunican los componentes padre e hijo en React?
63. ¿Qué función cumplen `npm` y `package.json` en un proyecto React?
64. ¿Qué es Vite y qué función cumple al crear un proyecto con React?
65. Explique la estructura básica de un proyecto React creado con Vite.
66. ¿Qué es React Router DOM y para qué se utiliza en una aplicación React?
67. ¿Qué son `import` y `export` en JavaScript y para qué se utilizan en un proyecto React?
68. ¿Qué es Axios y para qué se utiliza en una aplicación React?
69. ¿Cómo se realiza una petición `GET` utilizando Axios?
70. ¿Cómo se realiza una petición `POST` utilizando Axios?
71. ¿Qué es Node.js?
72. ¿Qué es una dependencia en un proyecto de software?
73. ¿Cuál es la diferencia entre una librería (biblioteca) y un framework?